1 Introduction to Sequence Modeling

1.1 What are Sequences?

Sequences are ordered collections of data where the order matters significantly. Unlike traditional machine learning problems where we assume data points are independent, sequence modeling requires understanding temporal or positional dependencies.

Common Sequence Types

Natural Language: "The cat sat on the mat" - Each word depends on previous words for meaning - Word order completely changes semantics
Time Series: Stock prices, weather data, sensor readings - Future values depend on historical patterns - Temporal correlations are crucial
Audio/Speech: Waveforms, phonemes, spoken words - Sequential structure encodes information - Context determines interpretation
DNA/Protein Sequences: Genetic codes, protein folding - Position-dependent biological functions - Long-range dependencies matter

1.2 Sequence-to-Sequence (Seq2Seq) Modeling

Definition 1. ***Sequence-to-Sequence modeling*** *maps an input sequence to an output sequence, potentially of different lengths:*

$$f : (x_1, x_2, \ldots, x_n) \to (y_1, y_2, \ldots, y_m)$$

Seq2Seq Applications

- **Machine Translation:** English → French
- **Summarization:** Long article → Summary
- **Question Answering:** Question + Context → Answer
- **Speech Recognition:** Audio → Text transcription
- **Image Captioning:** Image → Descriptive text
- **Code Generation:** Natural language → Programming code

1.3 Challenges in Sequence Modeling

Key Challenges

- **Variable Length:** Input/output sequences can vary dramatically
- **Long-range Dependencies:** Information from early tokens affects later ones
- **Sequential Processing:** Each step may depend on all previous steps
- **Computational Efficiency:** Processing long sequences is expensive
- **Gradient Flow:** Backpropagation through time can be unstable

2 Recurrent Neural Networks (RNNs)

2.1 Basic RNN Architecture

Recurrent Neural Networks were the dominant approach for sequence modeling until transformers emerged around 2017.

Definition 2. *A **Recurrent Neural Network** processes sequences by maintaining a hidden state that gets updated at each time step:*

$$h_t = tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h) \tag{1}$$
$$y_t = W_{hy}h_t + b_y \tag{2}$$

RNN Components

- x_t: Input at time step t
- h_t: Hidden state at time step t (captures "memory")
- y_t: Output at time step t
- W_{hh}: Hidden-to-hidden weight matrix
- W_{xh}: Input-to-hidden weight matrix
- W_{hy}: Hidden-to-output weight matrix
- b_h, b_y: Bias vectors

2.2 RNN Intuition

Think of an RNN as reading a book: - At each word (time step), you update your understanding (hidden state) - Your current understanding depends on the new word + what you understood before - You can make predictions based on your current understanding

RNN Example: Language Modeling

Input sequence: "The cat sat on the" **Goal:** Predict next word
Processing:

1. $h_1 = \tanh(W_{xh} \cdot \text{embedding}(\text{"The"}) + b_h)$
2. $h_2 = \tanh(W_{hh}h_1 + W_{xh} \cdot \text{embedding}(\text{"cat"}) + b_h)$
3. $h_3 = \tanh(W_{hh}h_2 + W_{xh} \cdot \text{embedding}(\text{"sat"}) + b_h)$
4. ... continue for each word
5. $y_5 = \text{softmax}(W_{hy}h_5 + b_y) \rightarrow$ probabilities over vocabulary

2.3 Training RNNs: Backpropagation Through Time (BPTT)

Training RNNs requires backpropagating gradients through the sequential computation graph.

Algorithm 1 Backpropagation Through Time

Forward Pass:
for $t = 1$ to T **do**
  $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$
  $y_t = W_{hy}h_t + b_y$
  Compute loss $L_t = \text{loss}(y_t, \text{target}_t)$
end for
Backward Pass:
Initialize $\frac{\partial L}{\partial h_T}$
for $t = T$ down to 1 **do**
  $\frac{\partial L}{\partial W_{hy}} += \frac{\partial L}{\partial y_t} h_t^T$
  $\frac{\partial L}{\partial h_t} = W_{hy}^T \frac{\partial L}{\partial y_t} + W_{hh}^T \frac{\partial L}{\partial h_{t+1}}$
  $\frac{\partial L}{\partial W_{hh}} += \frac{\partial L}{\partial h_t}(1 - h_t^2)h_{t-1}^T$
  $\frac{\partial L}{\partial W_{xh}} += \frac{\partial L}{\partial h_t}(1 - h_t^2)x_t^T$
end for

2.4 The Vanishing Gradient Problem

Vanishing Gradients in RNNs

When backpropagating through time, gradients must flow through repeated matrix multiplications:

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial h_T} \prod_{t=2}^{T} \frac{\partial h_t}{\partial h_{t-1}}$$

For simple RNNs: $\frac{\partial h_t}{\partial h_{t-1}} \approx W_{hh}\text{diag}(1 - h_{t-1}^2)$
If largest eigenvalue of W_{hh}:

- < 1: Gradients vanish exponentially $\rightarrow$ Can't learn long dependencies
- > 1: Gradients explode exponentially $\rightarrow$ Training instability

Gradient Analysis

For a simple case where $W_{hh} = w$ (scalar): - After T steps: gradient scales by w^T - If $|w| = 0.9$ and $T = 100$: scaling factor 3×10^{-5} (vanishing) - If $|w| = 1.1$ and $T = 100$: scaling factor 1.4×10^4 (exploding)

2.5 Long Short-Term Memory (LSTM)

LSTMs solve the vanishing gradient problem through gating mechanisms that control information flow.

Definition 3. ***LSTM*** *uses three gates and a cell state to selectively remember and forget information:*

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad \textit{(forget gate)} \tag{3}$$
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \textit{(input gate)} \tag{4}$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad \textit{(candidate values)} \tag{5}$$
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad \textit{(cell state)} \tag{6}$$
$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad \textit{(output gate)} \tag{7}$$
$$h_t = o_t * \tanh(C_t) \quad \textit{(hidden state)} \tag{8}$$

LSTM Gates Intuition

- **Forget Gate (f_t):** What information to discard from cell state
- **Input Gate (i_t):** What new information to store in cell state
- **Output Gate (o_t):** What parts of cell state to output
- **Cell State (C_t):** Long-term memory that flows with minimal modification

2.6 Advantages and Limitations of RNNs

RNN Advantages

- **Variable sequence length:** Can handle any input/output length
- **Parameter sharing:** Same weights used at all time steps
- **Theoretically powerful:** Can model any sequence pattern
- **Memory efficient:** Constant memory regardless of sequence length

RNN Limitations

- **Sequential processing:** Cannot parallelize training effectively
- **Vanishing gradients:** Difficulty learning long-range dependencies
- **Slow training:** Must process sequences step-by-step
- **Limited context:** Hidden state bottleneck for long sequences

3 Attention Mechanisms

3.1 Motivation for Attention

Traditional sequence-to-sequence models with RNNs encode the entire input sequence into a fixed-size vector (the final hidden state). This creates a bottleneck for long sequences.

Translation Problem

English: "The agreement on the European Economic Area was signed in August 1992." **French:** "L'accord sur la zone économique européenne a été signé en août 1992."
Problem: When translating "août" (August), the model needs to remember "August" from much earlier in the sentence. But this information might be lost in the fixed encoding.

3.2 Basic Attention Mechanism

Definition 4. ***Attention*** *allows the model to focus on different parts of the input sequence when producing each output:*

$$e_{i,j} = score(h_i, s_j) \quad \textit{(alignment scores)} \tag{9}$$

$$\alpha_{i,j} = \frac{\exp(e_{i,j})}{\sum_{k=1}^{T_x} \exp(e_{i,k})} \quad \textit{(attention weights)} \tag{10}$$

$$c_i = \sum_{j=1}^{T_x} \alpha_{i,j} h_j \quad \textit{(context vector)} \tag{11}$$

where: - h_j: encoder hidden state at position j - s_i: decoder hidden state at position i - c_i: context vector for decoder position i

8.2 Architecture Hyperparameters

Common Transformer Configurations

BERT-Base:

- Layers: 12, Hidden size: 768, Attention heads: 12
- Parameters: 110M

GPT-3:

- Layers: 96, Hidden size: 12,288, Attention heads: 96
- Parameters: 175B

Common ratios:

- $d_{ff} = 4 \times d_{model}$ (feed-forward dimension)
- $d_k = d_{model}/h$ (per-head dimension)

8.3 Data Preprocessing

Tokenization Strategies

- **Word-level:** Simple but large vocabulary
- **Character-level:** Small vocabulary but long sequences
- **Subword (BPE/SentencePiece):** Balance between vocabulary size and sequence length
- **Special tokens:** [PAD], [UNK], [CLS], [SEP], [MASK]

9 Mathematical Foundations

9.1 Attention as Matrix Operations

Self-attention can be viewed as matrix operations:

$$\text{Attention}(X) = \text{softmax}\left(\frac{XW^Q(XW^K)^T}{\sqrt{d_k}}\right)XW^V \tag{20}$$

$$= \text{softmax}\left(\frac{XW^QW^{K^T}X^T}{\sqrt{d_k}}\right)XW^V \tag{21}$$

Matrix Dimensions

For sequence length n and model dimension d:

- $X \in \mathbb{R}^{n \times d}$: Input embeddings
- $W^Q, W^K, W^V \in \mathbb{R}^{d \times d_k}$: Projection matrices
- $Q, K, V \in \mathbb{R}^{n \times d_k}$: Projected representations
- Attention weights: $\mathbb{R}^{n \times n}$
- Output: $\mathbb{R}^{n \times d_k}$

Sequence Models

- Sequence Models take **sequence of inputs** to a **specific output sequence**
  - E.g. Input: “The Cat in the” → Output: “Hat is a book by Dr. Seuss”
- **Versatile architecture** for learning about sequence data in any domain, where order matters, and where it can learn to predict, generate, classify or anything else

Language Model

- A **language model** is a **sequence model** trained specifically to capture the structure and probability distribution of language
- It can be seen **compact probability distribution** over sequences of tokens
  - Sequences are constructed from a vocabulary V, with tokens x_1,...,x_L, where each sequence has some probability between 0 & 1
- It is being modeled with the Markov Assumption, as that makes the problem computationally tractable
  - We couldn’t model the joint probability distributions of tokens appearing together, as that would make such a model computationally extremely infeasible

Joint Probability Distribution vs. conditional probability distribution

- ❗ We wouldn’t want to learn the **joint probability distribution** as that has some drawbacks
  →"What's the probability of this whole sentence"
  - It would be very hard to realize the bigger our vocabulary gets - computationally impossible?
  - When sampling, it would only give us the average sequence back, but not the “best” possible!
  - And we want to have varying length of generated sequences maybe, but cannot really control it?
- ✅ Instead, the **conditional probability distribution**, makes a lot more sense!
  → “Given the words so far, what's the most likely next word?”
  - We can use that to calculate the probability of getting a specific sequence of tokens!
  → $p(x_1 : L) = p(x_1) * p(x_2|x_1) * \ldots * p(x_L|x1 : L - 1)$

Architecture

Why are language models usually deep in terms of modules?

Many language & sequence models are usually just very deep, as you can have different modules with deeper understandings & different focus, so you can gain a better representation of the input and the next-token you want to predict

Encoder vs. Decoder

- **Encoder → Understanding** a sequence of tokens
  - **Bidirectional context →** To fully undestand a word’s meaning, you need the maximum surrounding context!
- **Decoder → Generating** sequences of tokens
  - **Unidirectional context →** When you write a sentence you **can’t see the words that come in the future**; it’s a causal process
    - During training, it would just learn to predict the next future word from the ground truth → undesired! Need to mask that!
- ❗ - BERT as a **famous encoder models**, GPT as a **Decoder-only model**

Decoder, Encoder, and what else? What is relevant?

Some sequence-to-sequence models can be decoder-encoder, encoder or decoder only - in the end it's just a matter of the problem context

GPT is **decoder-only**, but could work technically also with an encoder; reason is it's too *impractical*, as encoders have bidirectional causality, and each new chat message would need to go a full pass through the transformer, where a lot needs to recomputed & can't be reused! With only a *decoder & masked self-attention* we can reuse most of the computed vectors via KV-Cache

RNNs

RNNs work with processing specific input sequences **one by one, back to back**, and processing in a recurrent manner, taking the output of the previous sequence element when processing the next one. This allows for *variable input sequence lengths* and is computationally hence much more tractable & removes certain bias as well!

There are different variations of RNNs, where there is also bidirectionality involved and with techniques like LSTM/GRUs, e.g. to build stronger, more efficient and computational representations

Problems, Solutions & Variations (LSTM, GRU)

Long-range Dependencies (with Sequences)

The information of sequence elements at the beginning start to get less the further we are in the sequence, although they might be the most important!

(Motivation for an e.g. Attention Mechanism, that allows different sequence elements to "attend" to each other!)

Cannot be trained in parallel

- **Dependency chain by design**: An RNN computes $h_t = f(x_t, h_{t-1})$. That means ht *cannot be computed until $h{t-1}$* is known. So for a length-T sequence, the forward pass must run strictly in order t=1→T.

Vanishing/Exploding Gradients:

→ **A problem when reapplying the same weight matrix multiple times all over, that the gradient involves powers of the weights of the RNN

Based on whether the individual magnitude of the weight's elements are larger/smaller than 1, this can lead to exploding/vanishing gradients

**Solutions:

- **LSTM** can be a solution, where we have activation functions to limit ranges & some sort of "memory" to keep "important" information, and forget irrelevant stuff!
  - **GRU** (Gated recurrent units) are a bit simpler, and also do the similar!

LSTM

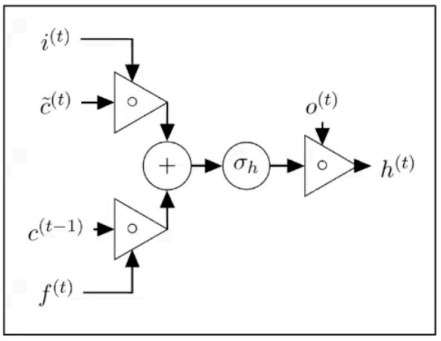
Long short term memory (LSTM)

- LSTM

(∘ denotes Hadamard product; σ is activation function)

$$
\begin{aligned}
i^{(t)} &= \sigma_i(W_{ix}x^{(t)} + W_{ih}h^{(t-1)} + w_{bi}) && \text{Input gate} \\
f^{(t)} &= \sigma_f(W_{fx}x^{(t)} + W_{fh}h^{(t-1)} + w_{bf}) && \text{Forget gate} \\
o^{(t)} &= \sigma_o(W_{ox}x^{(t)} + W_{oh}h^{(t-1)} + w_{bo}) && \text{Output/Exposure gate} \\
\tilde{c}^{(t)} &= \sigma_c(W_{cx}x^{(t)} + W_{ch}h^{(t-1)} + w_{bc}) && \text{New memory cell} \\
c^{(t)} &= f^{(t)} \circ c^{(t-1)} + i^{(t)} \circ \tilde{c}^{(t)} && \text{Final memory cell} \\
h^{(t)} &= o^{(t)} \circ \sigma_h(c^{(t)})
\end{aligned}
$$

- What do these gates do?
  - $i^{(t)}$: Does the current input $x^{(t)}$ matter?
  - $f^{(t)}$: Should the prior memory cell $c^{(t-1)}$ be forgotten?
  - $o^{(t)}$: How much of the current memory cell $c^{(t)}$ be used?
  - $c^{(t)}$: Compute new memory



Transformers

- **"Attention is all you need" -** Self-attention allows to circumvent to sequence based, recurring next-token prediction by allowing words to ***attend to each other***, irregardless of their positions and distance in the sequence

High-level structure

A transformer has different **blocks** (multiple dozens) that follow each other, where each block has different **attention heads**, where the attention mechanism runs in parallel with each having a different focus & context!

Example: "bank" in "I sat on the river bank" (represente as sequence of token embeddings X)

How does Attention work?

What is in an Attention Head?

- **Query (Q):** The "bank" token forms now a query vector with the Query Matrix of the specific attention head
  → Q = XW_Q
  - This vector encodes essentially, that this token is looking now for information related to e.g. geography, nature, sitting
- **Key (K):** Every other token in the sentence has now a **key vector,** where for each token we calculate the more specific "answer" it might have for the queries
  → K = XW_K
- **Value (V):** Every token also has a value vector, that is abstracting the meaning of that token, and that is also relevant to the context & specific area the attention head focuses in
  → V = XW_V
  → The **QKV** matrices in an attention head are intricately linked to each other. They are learned during training and are the core of what gives each attention head its unique "specialty

What happens?

1. With the query vector of "bank", we can now compare that to every other token's key vector, to find similarity through a dot product, giving us ***raw attention scores***

    1. This is hppening with a **causal mask**, so that words can only attend to previous words;
        1. This to allow for parallel training on a sequence of tokens without tokens being able to cheat, and look what follows
        2. By only being able to refer back, they learn predictive & generalizable relationships based on past context
2. After normalizing these attention scores via **softmax** we have for “bank” a distribution of how much different words get attended to
3. We then output a new vector representation for “token” based on adding & multiplying all the different value vectors of the relevant tokens

High-level Architecture

→ Standard Encoder-Decoder Transformer (although other variants do exist)

Be it Encoder or Decoder, both build on the same structure:

1. **Multi-Head Self-Attention**
    1. Here the difference it whether that happens bidirectionally or unidirectionally (unmasked vs. masked)
2. **Feed-Forward Network**
    1. Basic Neural Network that process each token’s representation somehow
3. **Residual Connections & Layer Normalization**
    1. For gradient flow reasons, we want to stabilize gradients & avoid stuff like e.g. vanishing gradients

Encoder vs. Decoder

1. The **Encoder Stack**
    1. Understanding the **whole input sequence** & building a deep representation via tokens
    2. Works with **bidirectional**, multi-headed self-attention
2. The **Decoder Stack**
    1. Generating an output sequence based on existing representations, one token at a time

Performance Considerations

KV-Cache

Why is the query irrelevant?

→ The KV Cache stores the context of the past, which is fixed. The Query is the question asked by the present, which is always new!